

SIMULAZIONE DI UN ECOSISTEMA DI FORMICHE

ant-sim

AUTORE: Naman Bagga

EMAIL: naman.bagga@studio.unibo.it

ANNO ACCADEMICO: 2025/2026

DATA INIZIO: 06/08/2026

DATA FINE: 10/08/2026

CAPITOLO 1

Analisi

Descrizione del progetto e requisiti

Il progetto ant-sim consiste in una simulazione di un ecosistema artificiale formato da una colonia di formiche che interagisce con l'ambiente, fonti di cibo e ostacoli presenti nel mondo. L'obiettivo è quello di rappresentare il comportamento di una colonia che esiste nel mondo reale, le formiche esplorano l'ambiente alla ricerca di fonti di cibo e, una volta scoperte, devono essere in grado di raccogliere e ritornare al nido trasportando il cibo. La comunicazione tra le formiche funziona tramite i feromoni e, il comportamento emergente delle formiche è ispirato ai principi della Ant Colony Optimization (ACO), dove le formiche utilizzano le informazioni presenti nell'ambiente per orientare il loro movimento e formare piste/tracce tra il nido e la fonte di cibo. La simulazione inoltre deve permettere di osservare graficamente l'evoluzione della colonia e far interagire l'utente con il mondo simulato.

Requisiti funzionali:

- Rappresentare l'ambiente come una griglia di celle;
- Rappresentare il contenuto delle celle come:
 - vuote;
 - ostacoli;
 - cibo;
 - nido.
- Rappresentare le formiche come agenti dotati di:
 - posizione;
 - orientamento;
 - velocità;
 - ruolo;
 - trasporto cibo/mani vuote.
- Permettere alle formiche di muoversi liberamente evitando eventuali ostacoli/bordi;
- Permettere alle formiche di individuare il cibo e di raccoglierlo/trasportarlo verso il nido;
- Permettere il deposito dei feromoni durante l'esplorazione e il trasporto di cibo verso il nido;
- Gestire i due tipi di feromone rilasciati:
 - cibo (per far conoscere la posizione della fonte anche alle altre formiche);
 - nido (per ricordarsi la posizione del nido).
- Gestire l'evaporazione dei feromoni nel tempo;
- Implementare una logica basata su ACO ma che non sia troppo rigida;
- Gestire il ciclo di vita della simulazione (stato iniziale, esecuzione, pausa ed ecc...);
- Permettere di visualizzare la simulazione e i suoi elementi (tramite javafx):
 - formiche, cibo, nido e ostacoli;
 - deposito e evaporazione di feromoni;
 - Statistiche della simulazione (che possono essere eventualmente cambiate);
- Permettere all'utente di creare il suo scenario disegnando.

Modello del dominio

Il dominio della simulazione è composto da un ambiente discreto su cui le formiche possono muoversi in modo continuo (dotate di una posizione continua). Infatti, l'ambiente è rappresentato da una griglia bidimensionale ove ogni cella può essere di 4 tipi:

- Vuota o ostacolo;
- Più fonti di cibo e un solo nido;

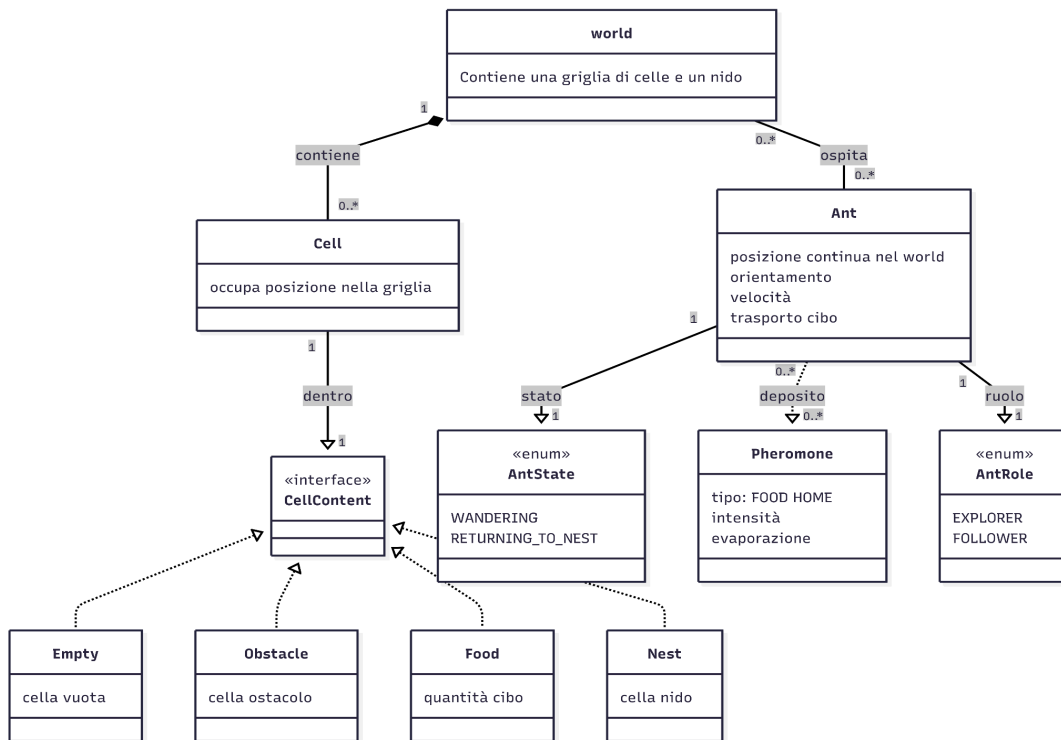
In sostanza, la griglia rappresenta l'elemento discreto della simulazione mentre le formiche si muovono su questa griglia sfruttando le coordinate continue. Una formica è un agente autonomo che possiede posizione, velocità ed ecc.. Il suo comportamento è determinato dal ruolo e dal suo stato. Sono presenti due ruoli principali:

- Explorer, formiche che favoriscono l'esplorazione;
- FOLLOWER, formiche che favoriscono le informazioni sulle piste.

Durante la simulazione, una formica può trovarsi in due stati differenti a seconda del scenario:

- Wandering, rappresenta la ricerca del cibo;
- RETURNING_TO_NEST, rappresenta il trasporto di cibo verso il nido.

Il comportamento collettivo è supportato dai feromoni, per esattezza sono presenti due tipi di feromoni ovvero HOME e FOOD. Questi permettono di distinguere le tracce associate ai percorsi principali della colonia. Il nido costituisce un riferimento di riferimento verso il quale le formiche devono tornare. Quando una formica raggiunge il nido, deposita il cibo trasportato e ritorna in stato di Wandering. Le fonti di cibo sono quantitative e limitate per cella e questa quantità viene ridotta ogni volta che una formica raccoglie una unità.



Schema UML che riassume la fase di analisi del dominio, in questa figura sono rappresentate le componenti principali e le loro relazioni.

CAPITOLO 2

Design

Architettura

Il progetto è stato realizzando seguendo il pattern MVC (Model, View e Controller) in modo da separare le responsabilità e non fare un miscuglio tra i vari componenti.

Model (SimulationEngine)

SimulationEngine è colui che coordina l'intero stato del dominio:

- World (la griglia);
- AntGroup (le formiche);
- PheromoneField (campo di feromoni);
- DecisionEngine (logica per la decisione).

Inoltre, espone lo stato tramite SimulationStatistics e le operazioni del ciclo della simulazione:

- start (inizio simulazione);
- pause (pausa simulazione);
- resume (riprendere la simulazione dopo che è stata messa in pausa);
- stop (fermare tutto);
- step (avanzare la simulazione);
- reset (restart della simulazione);
- createEmptyScenario (creazione scenario vuoto per la simulazione);

- generateScenario (generazione scenario casuale);
- addAnt (aggiungere una formica);
- setAgentCount (portare il numero di formiche al valore richiesto);
- foodPickup (raccogliere cibo);
- nestDelivery (consegna cibo);
- getWorld (mondo corrente);
- getAnts (lista formiche attive);
- getStatus (stato corrente simulazione);
- getClock (orologio interno della simulazione);
- getStats (statistiche);

Non conosce né il controller né la view.

View (SimulationView)

SimulationView disegna/mostra la simulazione:

- griglia;
- formiche;
- statistiche;
- feromoni;

Inoltre gli eventi dell'utente al controller tramite i callback:

- setOnStartPause (utente preme pulsante avvia/pausa);
- setOnReset (utente preme pulsante di reset);
- setOnAntCountChanged (utente modifica numero formiche);
- setOnSpeedChanged (utente modifica velocità simulazione);
- setOnCanvasResized (area di disegno ridimensionata);
- setOnGenerateWorld (utente richiede generazione scenario casuale);
- setOnWorldEdit (utente modifica personalmente lo scenario);

La view non comunica direttamente con il model ma sfrutta il controller attraverso i callback appena citati. Inoltre espone un metodo per diffondere lo stato sui controlli grafici attraverso setRunning(boolean running).

Controller (SimulationController)

SimulationController è il ponte tra SimulationEngine (il model) e SimulationView (la view) e ha tre responsabilità/compiti:

1. Collegamento tra View e Model

Il controller riceve un `SimulationEngine` e crea dentro di sé la `SimulationView`, per poi registrarsi come destinatario dei callback provenienti dalla View e associando un metodo ad ognuno dei callback:

<code>setOnStartPause</code>	<code>toggleSimulation()</code>
<code>setOnReset</code>	<code>resetSimulation()</code>
<code>setOnAgentCountChanged</code>	<code>setAntCount(count)</code>
<code>setOnSpeedChanged</code>	aggiorna il campo <code>speedMultiplier</code>
<code>setOnCanvasResized</code>	<code>render()</code>
<code>setOnGenerateWorld</code>	<code>generateScenario()</code>
<code>setOnWorldEdit</code>	<code>applyWorldEdit(edit)</code>

Ogni volta che l'utente interagisce con la UI, la View si limita a invocare i callback ed è proprio il controller che decide cosa fare.

2. Scandire il tempo della simulazione

Il controller ha una `Timeline` di `JavaFX` per generare un tick ogni 33 ms (30 fotogrammi per secondo) attraverso il metodo `tick()` che si comporta nel seguente modo:

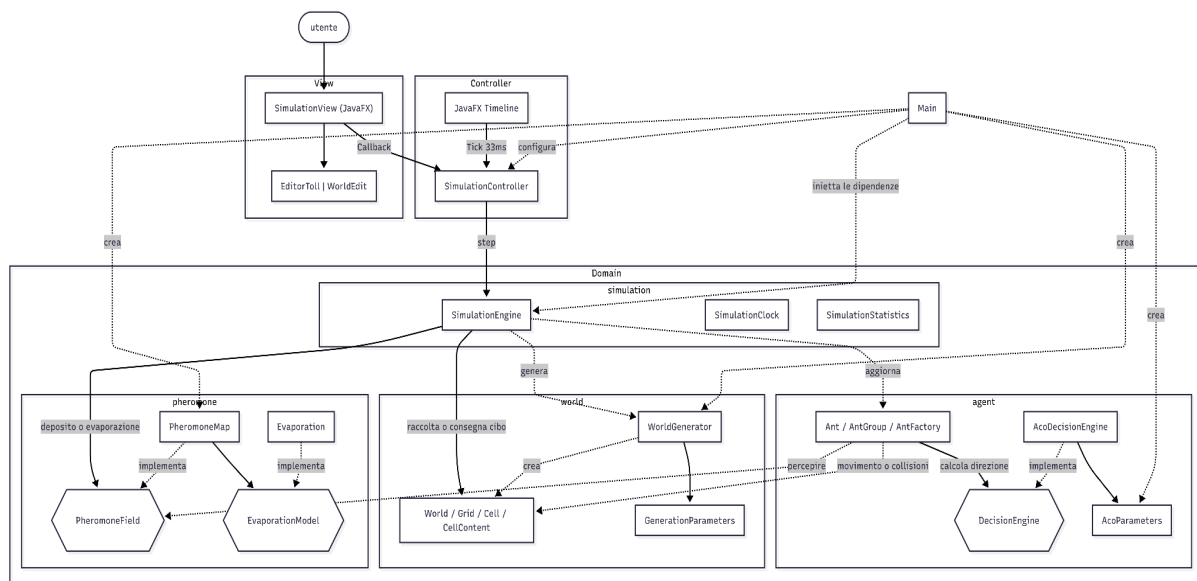
- se la simulazione non è in `RUNNING`, non fa nulla;
- nel caso in cui lo sia, calcola il passo da far avanzare moltiplicando un passo di 0.05 s per il moltiplicatore della velocità che viene importato dall'utente tramite lo slider;
- chiama poi `engine.step(dt)` facendo avanzare la simulazione di quel passo;
- e infine chiama `render()` per aggiornare lo stato visivo.

3. Traduzione eventi utente

I metodi collegati ai callback (vedere la tabella sopra) traducono un singolo evento in una o molteplici chiamate al Model, e se c'è bisogno di un aggiornamento dello stato visivo viene chiamato il metodo `render()`.

Il Model (package `simulation`, `agent`, `world` e `pheromone`) non ha dipendenze da `JavaFX`, dal controller o dalla view. Infatti è possibile utilizzarlo con una View differente senza modificare il codice.

Il controller, ha solamente una dipendenza dalla `Timeline` di `JavaFX` però sostituendo la View con altro si dovrebbe adattare il controller alla nuova View.



Schema UML che riassume l'architettura del progetto.

Design dettagliato

Algoritmo decisionale

Problema

Un movimento totalmente casuale non permetteva di produrre un percorso stabile ed efficiente. Era quindi necessario introdurre un sistema in grado di sfruttare le informazioni presenti nell'ambiente senza rendere troppo rigido il comportamento degli agenti. Inoltre bisognava tenere conto della presenza di ostacoli nella griglia.

Soluzione

Per risolvere questo problema è stata introdotta l'interfaccia DecisionEngine, ovvero il sistema secondo il quale una formica sceglie il suo orientamento, che definisce

decideNextangle(Ant, World, PheromoneField) mentre, l'implementazione vera avviene in AcoDecisionEngine. Ad ogni chiamata, AcoDecisionEngine valuta 3 direzioni candidate di fronte alla formica:

- direzione centrale;
- direzione a sinistra;
- direzione a destra.

L'algoritmo usato si ispira alla formula classica ACO che calcola la probabilità che una formica k scelta di spostarsi da un nodo a un altro:

$$P_{ij}^k = \frac{(\tau_{ij})^\alpha \cdot (\eta_{ij})^\beta}{\sum_{K \in ALLOWED_K} (\tau_{ik})^\alpha \cdot (\eta_{ik})^\beta}$$

- P_{ij}^k : Rappresenta la probabilità che la formica k , trovandosi nel nodo i , scelga di spostarsi nel nodo j .
- τ_{ij} (**Tau**): Indica la quantità di scia chimica presente sul collegamento tra il nodo i e il nodo j . Più è alta, più il percorso è stato storicamente preferito da altre formiche.
- η_{ij} (**Eta**): Rappresenta la desiderabilità locale. Più il nodo j è vicino al nodo attuale i , più questo valore è alto.
- α (**Alpha**): Regola l'importanza della scia di feromone. Se $\alpha = 0$, le formiche ignorano la storia passata e scelgono solo in base alla vicinanza.
- β (**Beta**): Regola l'importanza della vicinanza geometrica. Se $\beta = 0$, le formiche seguono ciecamente il feromone, rischiando di bloccarsi su percorsi non ottimali.
- $ALLOWED_K$: Contiene tutti i nodi vicini che la formica k non ha ancora visitato nel suo viaggio attuale (evitando così che la formica ritorni su una città già toccata).

La formula sopra citata è pensata per un grafo discreto in cui una formica sceglie fra i nodi vicini, ma la simulazione non lavora su un grafo. Infatti le formiche si muovono con coordinate continue su una griglia, quindi la formula è stata rivisitata nel seguente modo:

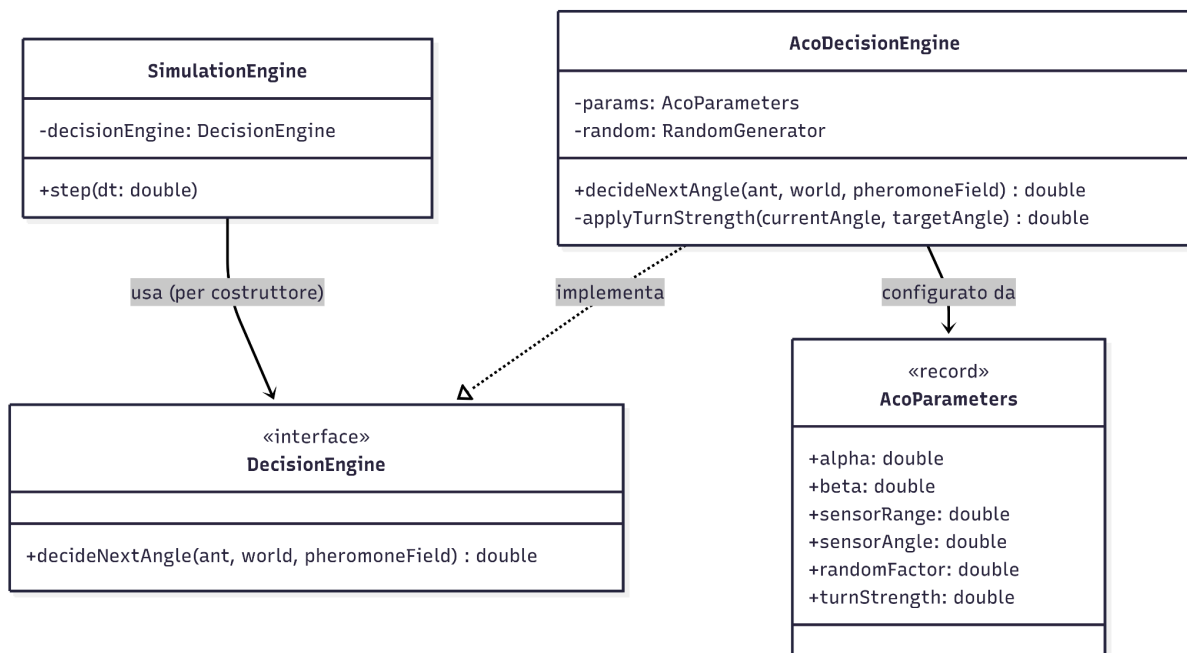
1. al posto di $ALLOWED_K$ ci sono sempre tre direzioni candidate, citate sopra;
2. l'euristica è un valore fisso che dipende dalla direzione (1.0 per il centro e 0.5 per i lati);
3. al peso ACO si aggiungono un tasso di esplorazione (explorationRate) e, solo quando la formica sta tornando verso il nido, un fattore di homing per aiutarle a tornare al nido (verrà spiegato nel prossimo problema).

$weight[i] = (1 - explorationRate) \cdot (pheromone + \epsilon)^\alpha \cdot heuristic^\beta + explorationRate$

- pheromone: livello di feromone campionato in direzione;

- ϵ (epsilon): piccola costante per garantire un peso non nullo in assenza di feromone;
- heuristic: i valori fissi citati sopra per l'euristica.

Infine viene calcolata la probabilità di scegliere una direzione e la scelta finale viene fatta attraverso un algoritmo noto per quanto riguarda gli input pesati, ovvero la roulette-wheel selection. Infatti, viene estratto un numero tra l'intervallo $[0, \text{totalWeight})$, dove totalWeight è la somma dei pesi delle direzioni candidate, e si seleziona la prima direzione la cui somma cumulativa è maggiore rispetto al numero casuale. Inoltre è presente anche una piccola probabilità, explorationProb , che permette di ignorare del tutto i pesi e scegliere una direzione a caso per evitare che una colonia diventi troppo rigida. Nel caso in cui tutte le direzioni sono bloccate, viene scelta una direzione casuale per evitare il blocco.



Ritorno al nido

Problema

Le formiche dopo aver trovato la fonte di cibo e formato il percorso, non ritornavano al nido. Tendevano a formare i cosiddetti anelli della morte, percorsi che schivavano la posizione del nido o semplicemente percorsi che non tornavano al nido. Il solo meccanismo di lettura dei feromoni non garantiva un ritorno affidabile.

Soluzione

Quando una formica si trova in `RETURNING_TO_NEST`, **AcoDecisionEngine** calcola ad ogni decisione, `homingAngle` ovvero un angolo che punta dalla posizione corrente della formica alla posizione centrale del nido attraverso la seguente formula:

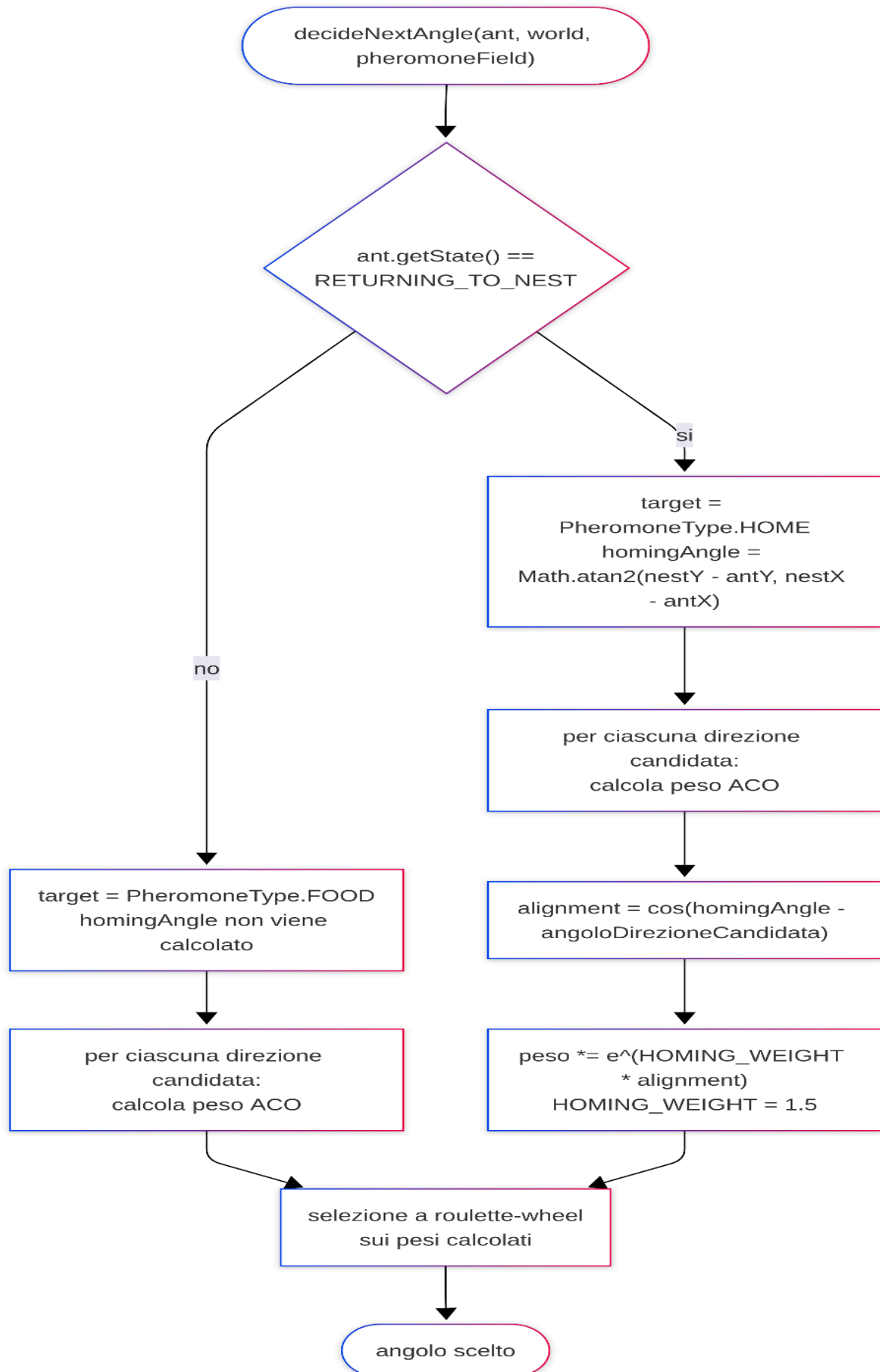
`Math.atan2(nestY - antY, nestX - antX)`

Per ciascuna delle tre direzioni candidate, viene calcolato quanto siano allineata con homingAngle tramite $\text{alignment} = \cos(\text{homingAngle} - \text{angoloDirezioneCandidata})$ e vale:

- 1 se la direzione coincide con quella del nido;
- -1 se è opposta.

Questo allineamento serve a amplificare il peso già calcolato del feromone, infatti non sostituisce il comportamento di seguire il feromone. Se la pista di feromoni è forte e allineata col nido i due segnali di rinforzano e se la pista è debole o assente, è comunque presente una spinta verso il nido.

Era stato considerato di usare solamente un feromone ovvero quello depositato appena le formiche escono dal nido, ma l'idea è scartata per disorientamento.



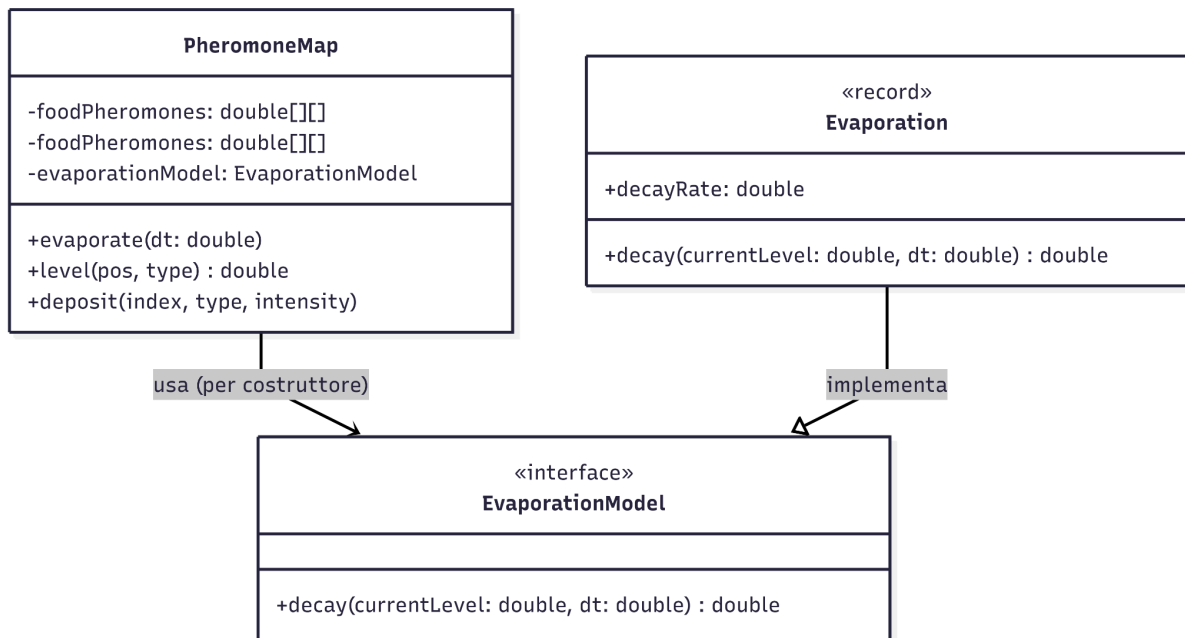
Modello di evaporazione

Problema

L'evaporazione inizialmente era lineare, ma poteva produrre valori negativi se non gestita bene e inoltre bisognava tenere un modello di esplorazione separato, in modo da riuscire a sostituirlo senza toccare altri file.

Soluzione

Anche qui si applica una soluzione simile a quella del comportamento decisionale delle formiche, ovvero avere una interfaccia `EvaporationModel` con il metodo `decay(currentLevel, dt)`, avere un record `Evaporation` che lo implementa e `PheromoneMap` riceve l'`EvaporationModel` per costruttore e lo invoca su tutte le celle della griglia ad ogni passo senza sapere come viene calcolato.



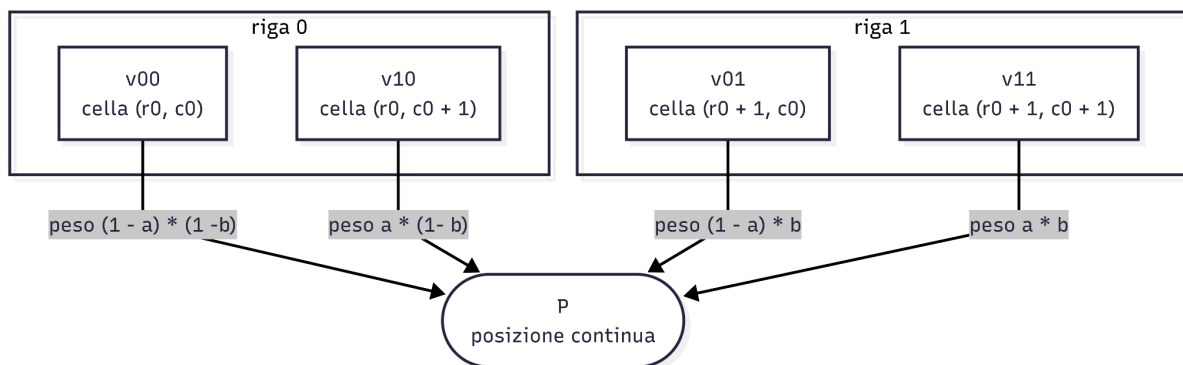
Rappresentazione campo feromoni

Problema

Le formiche hanno una posizione continua, ovvero sfruttano coordinate continue per muoversi, mentre i feromoni sono definiti su una griglia discreta (una cella -> un valore).

Soluzione

PheromoneMap ha due griglie parallele di valori double, foodPheromones e homePheromones (come si può notare dal diagramma UML della problematica antecedente) che hanno la stessa dimensione della griglia. Il metodo level(WorldPosition pos, PheromoneType type) calcola le 4 celle che circondano la posizione continua e restituisce l'interpolazione bilineare dei loro valori in base a quanto la posizione è vicina a ciascuna cella. In questo modo il campo percepito varia in modo continuo anche se la struttura resta discreta.



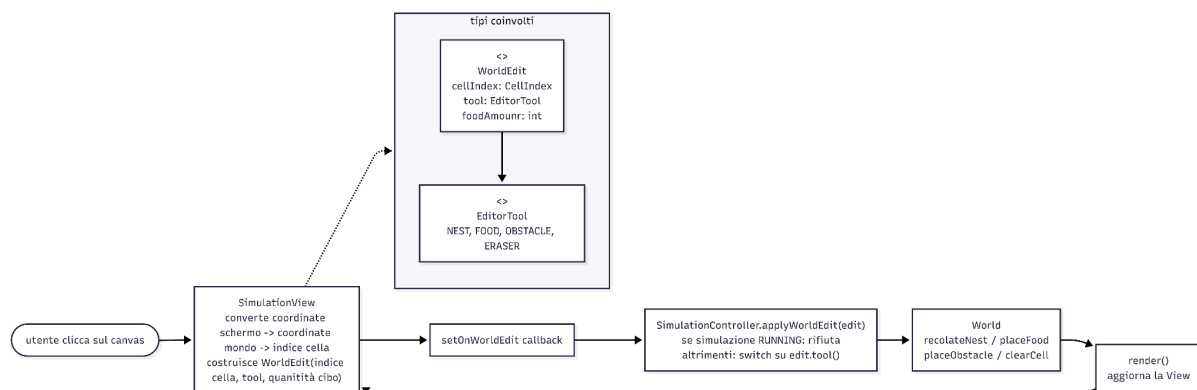
Editor interattivo

Problema

L'utente deve poter disegnare uno scenario senza che la View modifichi direttamente lo stato del Model.

Soluzione

L'interazione dell'utente viene modellata con l'enum EditorTool(NEST, FOOD, OBSTACLE, ERASER) e con il record WorldEdit(CellIndex index, EditorTool tool, int foodAmount). Quanto l'utente clicca sul canvas, la View converte le coordinate dello schermo in coordinate del mondo e poi nella cella corrispondente, successivamente passa ogni callback al Controller che nel metodo ApplyWorldEdit, decide quale operazione rifiutare o far fare.



CAPITOLO 3

Sviluppo

Testing automatizzato

il progetto usa JUnit per i test automatici che coprono tutti i livelli del dominio (senza dipendere da JavaFX).

world

- GridTest: verifica l'inizializzazione della griglia, l'accesso/modifica delle celle, i limiti e gli errori fuori dai confini.
- WorldTest: verifica le operazioni sul mondo, inclusi spostamento del nido, vicini percorribili, conversione coordinate e ricerca del cibo.
- CellTest: verifica il contenuto iniziale e le transizioni di una cella, compreso il consumo completo del cibo.
- CellContentTest: verifica la validazione e il consumo delle quantità di cibo.
- WorldPositionAndCellIndexTest: verifica la correttezza degli oggetti valore usati per coordinate continue e indici discreti delle celle.
- WorldGenerationTest: verifica che gli ostacoli rispettino l'area libera attorno al nido e che la generazione sia riproducibile a seed fisso.

agent

- AntMovementTest: verifica il movimento continuo in spazio libero e il comportamento in presenza di muri/ostacoli.
- AntStateTest: verifica lo stato iniziale dell'agente, le transizioni raccolta/consegna e il rifiuto delle transizioni non valide.
- AntGenerationTest: verifica che le formiche generate siano posizionate all'interno della cella del nido.
- AcoDecisionEngineTest: verifica che l'algoritmo favorisca direzioni con più feromone e rispetti il limite di sterzata.

pheromone

- PheromoneMapTest: verifica la saturazione dei feromoni, l'interpolazione spaziale dei valori e l'evaporazione esponenziale.

simulation

- SimulationEngineLifecycleTest: verifica lo stato iniziale del motore e le transizioni valide/non valide del suo ciclo di vita.
- SimulationStepTest: verifica che un passo avanzi il tempo, muova gli agenti e attivi l'evaporazione dei feromoni.
- SimulationFoodPickupTest: verifica che la quantità di cibo raccolta sia proporzionale alle formiche coinvolte.
- SimulationNestDeliveryTest: verifica che la consegna al nido incrementi il cibo accumulato, reimposti lo stato della formica e depositi feromone durante il ritorno.
- SimulationAgentCountTest: verifica l'aumento/riduzione del numero di agenti e il rifiuto di valori negativi.
- SimulationResetTest: verifica che il reset rigeneri il mondo e azzeri lo stato della simulazione.

- `SimulationConvergenceTest`: verifica che lo stesso seed produca lo stesso esito e che la simulazione raggiunga uno stato stazionario entro il numero massimo di passi.

Note di sviluppo

Record

Sono stati utilizzati record per rappresentare dati puramente strutturale e immutabili come ad esempio:

- `WorldPosition` [\[PERMALINK GITHUB\]](#);
- `CellIndex` [\[PERMALINK GITHUB\]](#);
- `AcoParameters` [\[PERMALINK GITHUB\]](#);
- `SimulationStatistics` [\[PERMALINK GITHUB\]](#).

Sealed interface

Viene utilizzata in `CellContent` per modellare il contenuto di una cella come unione chiusa di 4 varianti (`Empty`, `Obstacle`, `Food`, `Nest`) e soprattutto per modifiche sicure.

[\[PERMALINK GITHUB\]](#)

instanceof

Usato ad esempio in `World.consumeFood` per estrarre e verificare solo il contenuto della cella in un colpo solo.

[\[PERMALINK GITHUB\]](#)

Optional

Per esprimere risultati che potrebbero non essere presenti, come ad esempio al ricerca di cibo nelle vicinanze.

[\[PERMALINK GITHUB\]](#)

RandomGenerator

Viene utilizzato in `AntFactory` e `WorldGenerator` per rendere il comportamento riproducibile.

[\[PERMALINK GITHUB\]](#)

[\[PERMALINK GITHUB\]](#)

Codice derivato da fonti esterne e utilizzo AI

il progetto è stato sviluppato individualmente, nonostante all'inizio era un progetto suddiviso tra 2 componenti. Durante alcune fasi dello sviluppo sono stati utilizzati strumenti di intelligenza artificiale come supporto al planning, all'analisi dei problemi implementativi e al confronto tra le soluzioni possibili.

L'utilizzo di tale strumento ha riguardato casi quali presentavano un blocco o che richiedevano una scelta progettuale che cambiava l'intera struttura del progetto, in particolare è stata utilizzata come strumento di supporto per:

- analizzare strategie per il ritorno al nido;
- ragionare sul comportamento ACO e sulla combinazione tra esplorazione, feromoni e orientamento (soprattutto per quanto riguarda i parametri che possono essere modificati nel codice);
- valutare approcci per la realizzazione dell'interattività della GUI in JavaFX;
- discutere possibili refactoring di codice per migliorare la sintassi o per migliorare il funzionamento;
- confrontare diverse alternative implementative.

Non è stato utilizzato un progetto esistente come base da modificare.

CAPITOLO 4

Commenti finali

Autovalutazione e lavori futuri

Il progetto è stato sviluppato comprendendo analisi, progettazione, testing, debugging, integrazione GUI e refactoring. Probabilmente uno dei principali punti di forza di questo progetto è la separazione delle responsabilità sfruttando il pattern MVC. Un ulteriore punto di forza è la presenza di test automatici che coprono quasi del tutto il sistema e non presentano dipendenze da JavaFX.

Probabilmente la parte che ha richiesto maggiore lavoro è stata la realizzazione del comportamento ACO, integrazione di JavaFX e il refactoring finale per la build.

Tra i possibili lavori futuri ci potrebbero essere:

- usare il multithreading per le formiche, ora sono su single thread e un elevato numero fa girare in modo scattoso le formiche;
- migliorare l'interfaccia grafica;
- permettere a più colonie di essere presenti nella simulazione;
- avere diversi ruoli di formiche e altri tipi di feromone per le formiche per completezza;

Difficoltà incontrate

Sicuramente la non formazione dei percorsi ottimali ma dei cosiddetti cerchi della morte. Questi cerchi della morte, succedevano quando una formica, che aveva appena raccolto il cibo, si affida solo esclusivamente al feromone per orientarsi verso il nido. Il risultato era che le formiche, al posto di dirigersi verso il nido, giravano su se stesso oppure seguivano piste che si allontanavano dal nido senza toccarlo minimamente. Se una formica tornava al nido, era per pura fortuna.

Un'ulteriore difficoltà è stata ritrovarsi a fare il progetto individualmente dopo che si era formato il gruppo. Certo, nessun odio o nessun rancore però era meglio essere avvisati prima piuttosto che quando la deadline del progetto era al termine.

APPENDICE A

Guida per l'utente

All'avvio, l'applicazione (Main) apre una finestra intitolata AntSim (1280×800 px, ridimensionabile fino a un minimo di 950×600 px) divisa in tre zone: una barra superiore con i comandi principali, un canvas centrale su cui viene disegnata la simulazione, e un pannello laterale destro con statistiche, editor della mappa e parametri.

Avvio: mondo iniziale vuoto

Alla prima apertura il mondo è completamente vuoto: nessun nido, nessun ostacolo, nessuna fonte di cibo. Prima di poter avviare la simulazione è quindi necessario generare uno scenario, con una delle due modalità descritte sotto.

Barra superiore

- Avvia / Pausa, avvia la simulazione se è ferma o in pausa; la mette in pausa se è in esecuzione. L'etichetta del pulsante cambia di conseguenza. Se il nido non è ancora stato posizionato, la simulazione non può partire.
- Genera scenario, crea un mondo casuale di 80×100 celle, con cluster di ostacoli e di cibo generati proceduralmente attorno a punti casuali, e il nido posizionato al centro della griglia.
- Mappa vuota, svuota completamente il mondo, per ricominciare a disegnare uno scenario manualmente con l'editor.

Pannello laterale — STATISTICHE

Mostra in tempo reale, aggiornate ad ogni tick della simulazione:

- lo stato corrente (Stato: IDLE/RUNNING/PAUSED/STOPPED)
- il numero di passi eseguiti (Step), il tempo simulato trascorso in secondi (Tempo)
- il numero di formiche attive (Formiche) e la quantità totale di cibo raccolto (Cibo raccolto).

Pannello laterale — EDITOR MAPPA

Permette di disegnare manualmente lo scenario tenendo premuto e trascinando il mouse sulla canvas. Sono disponibili quattro strumenti, uno selezionabile alla volta:

- NIDO, sposta il nido nella cella cliccata (selezionato di default);
- CIBO, posiziona una fonte di cibo, con quantità configurabile tramite lo spinner QUANTITA' CIBO subito sotto (valore tra 1 e 100.000, di default 1.000);
- OSTACOLO, posiziona un ostacolo;
- CANCELLA, svuota la cella cliccata.

L'editor funziona solo quando la simulazione non è in esecuzione: se si prova a disegnare mentre le formiche si muovono, la modifica viene ignorata.

Pannello laterale — PARAMETRI

- Numero Formiche, spinner che imposta il numero di formiche attive, da 1 a 25.000. Le nuove formiche vengono generate all'interno della cella del nido (AntFactory).

- Velocità, slider che regola il moltiplicatore di velocità della simulazione (speedMultiplier, tra 0.25× e 5×, default 1×): determina quanto tempo simulato avanza ad ogni tick reale di 33 ms.

Visualizzazione sul canvas

Il canvas mostra, in ordine di disegno: i due campi di feromoni (FOOD e HOME) come sovrapposizioni colorate sulla griglia, poi la griglia con ostacoli, fonti di cibo e nido, e infine le formiche. Ridimensionando la finestra, il disegno si adatta automaticamente alle nuove dimensioni della canvas.